

Medical RAG Chatbot — Complete Feature & Implementation Reference

A production-grade AI assistant that answers questions about your gut-health and microbiome lab reports using Retrieval-Augmented Generation.

PRODUCT	BACKEND	FRONTEND	DEPLOYMENT
GutBot / IOM BioWorks	Flask · FAISS · SentenceTransformers · AWS Bedrock	React 19 · Vite · Axios	AWS EC2 · Nginx · Gunicorn · systemd
DOCUMENT DATE	STATUS		
May 2026	Production-Ready		

Table of Contents

OVERVIEW

- 1. Project Overview & Purpose
- 2. Technology Stack
- 3. System Architecture

CORE FEATURES

- 4. PDF Upload & Processing Pipeline
- 5. Medical Document Validation Layer
- 6. RAG Chat Engine
- 7. Inference Engine & Query Routing
- 8. Security Guard
- 9. Multi-Document Management
- 10. Conversation Isolation

FRONTEND

- 11. Frontend Application Features
- 12. API Client & Request Layer

API REFERENCE

- 13. REST API Endpoints

DEPLOYMENT

- 14. Local Development Setup
- 15. EC2 Production Deployment
- 16. Configuration Reference

AUDIT & CHANGES

- 17. Bug Fixes Implemented
- 18. Security Hardening
- 19. Production Readiness Assessment
- 20. Known Limitations & Future Work

1. Project Overview & Purpose

GutBot is a production-grade AI chatbot built for **IOM BioWorks**. It allows patients and clinicians to upload medical PDF documents — gut-health reports, microbiome analyses, blood panels, discharge summaries — and ask natural-language questions about them. The system retrieves the most relevant passages from the document and passes them to a large-language model (AWS Bedrock) to generate a grounded, cited answer.

What makes GutBot different from a generic chatbot

- **Document-grounded answers:** Every response is rooted in passages from the user's own uploaded PDF, not generic web knowledge.
- **Per-user session isolation:** Each browser tab has a unique `conversationId`. Documents uploaded in one session are completely invisible to every other session.
- **Medical-only gate:** A multi-signal heuristic validator rejects non-medical PDFs before they enter the processing pipeline — no invoices, legal contracts, or code files can be indexed.
- **Smart query routing:** An inference engine classifies every question before touching FAISS. Greetings, out-of-domain questions, and general medical queries are answered directly by the LLM without a vector search, saving latency and cost.

Primary use cases

- Patient wants to understand their IOM SEnS / IBS microbiome report
- Clinician wants to quickly query a discharge summary or blood panel
- Health professional comparing two lab reports from the same patient

2. Technology Stack

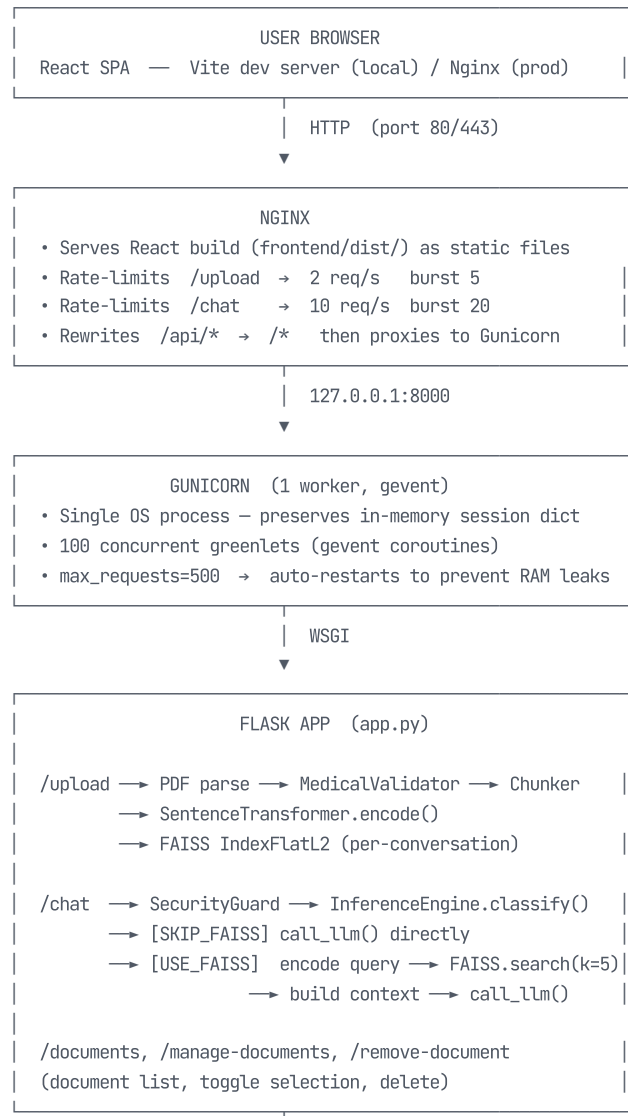
Layer	Technology	Version / Notes
Backend framework	Flask	Python 3.10+
Vector store	FAISS (Facebook AI)	In-memory, per-conversation index
Embeddings	SentenceTransformers	all-MiniLM-L6-v2 — 384-dim, runs on CPU or GPU
LLM backend	AWS Bedrock	Mistral 7B Instruct v0.2 (default) or Claude Haiku
PDF parsing	PyPDF2	Text extraction only
CORS	flask-cors	Production whitelist, dev wildcard
Frontend	React 19 + Vite	SPA with react-markdown, axios, uuid, date-fns
HTTP server	Gunicorn + gevent	1 worker, 100 greenlets — required for session isolation
Reverse proxy	Nginx	Rate limiting, gzip, static file serving
Process manager	systemd	Auto-restart, CPU/RAM quotas, SSH-safe
Cloud platform	AWS EC2	Ubuntu 22.04 LTS — t3.large or t3.xlarge recommended

AWS Bedrock Model Options

Model ID	Speed	Quality	Recommended for
<code>mistral.mistral-7b-instruct-v0:2</code>	Fast	Good	Default — best cost/speed balance
<code>anthropic.claude-haiku-4-5-20251001-v1:0</code>	Very fast	Very good	High-volume, cost-sensitive
<code>anthropic.claude-sonnet-4-5-20251001-v1:0</code>	Medium	Excellent	Premium quality responses
<code>meta.llama3-8b-instruct-v1:0</code>	Medium	Good	Open-weight alternative

3. System Architecture

High-level request flow





AWS BEDROCK	
Mistral 7B / Claude Haiku	(configurable via .env)
Region: ap-south-1	(configurable)

Session isolation design

```
conversation_indices = {
  "uuid-alice-session": {
    "documents": {
      "doc-uuid-1": { chunks: [...], filename: "report.pdf", selected: True },
    },
    "combined_index": <faiss.IndexFlatL2>,
    "doc_to_chunk_mapping": { 0: ("doc-uuid-1", 0), 1: ("doc-uuid-1", 1), ... }
  },
  "uuid-bob-session": { ... }, ← completely separate
}
```

CRITICAL ARCHITECTURE CONSTRAINT

Gunicorn **must run with exactly 1 worker process**. `conversation_indices` is a plain Python dictionary in process memory. Multiple Gunicorn workers would be separate OS processes with separate memory — User A's FAISS index would silently disappear when User B's request lands on a different worker. Concurrency is handled by gevent coroutines (non-blocking I/O) within the single worker.

Frontend directory structure

```
frontend/
├─ src/
│  ├─ App.jsx           ← root component, conversation + upload logic
│  ├─ App.css
│  ├─ main.jsx          ← React entry point
│  ├─ index.css
│  └─ components/
│     ├─ ChatContainer.jsx ← renders message history
│     └─ ChatInput.jsx    ← send message, keyboard shortcut
```



```

| | | DocumentManager.jsx ← document list, toggle, remove
| | |─ Header.jsx
| | |─ LoadingSpinner.jsx
| | |─ Sidebar.jsx ← conversation list
| | |─ UploadZone.jsx ← drag-drop or click upload
| | |─ ErrorBoundary.jsx ← React error boundary
| |─ services/
| | |─ apiClient.js ← primary API client (dedup, retry, cache)
| |─ utils/
| | |─ api.js ← legacy axios helpers
| | |─ helpers.js
| | |─ storage.js
| | |─ validation.js
|─ .env ← VITE_API_URL (local dev)
|─ .env.production ← VITE_API_URL=/api (production build)
|─ vite.config.js ← dev proxy → localhost:8000
|─ package.json

```

Backend directory structure

```

virviai-iom-bioworks-gutbotprod/
├─ backend/
|   │ app.py ← Flask application (main entry point)
|   │ llm_config.py ← LLM backends, InferenceEngine, SecurityGuard
|   │ medical_doc_validator.py ← Medical document validator (NEW)
|   │ requirements.txt
|   │ start.sh ← launch script
|   │ .env ← credentials (never commit)
|   │ .env.example ← template
|   │ uploads/ ← uploaded PDFs (temporary)
|   │ indices/ ← pre-built static FAISS indices
|   │ data/
|   │ │ processed/ ← JSON chunk files
|   │ │ └─ raw/ ← raw Excel/JSON source data
|   │ └─ build_*.py ← index-building scripts (offline)
├─ ec2-files/
|   │─ gunicorn.conf.py ← Gunicorn production config
|   │─ gutbot.service ← systemd service unit
|   │─ nginx.conf ← Nginx main config (SSH-safe)
|   │─ nginx-gutbot ← Nginx site config (rate limits, routing)
|   │─ deploy.sh ← one-shot deployment script
|   │─ SETUP.md ← pre-flight checklist
├─ README.md
├─ ARCHITECTURE.md

```

```
|— RUN_GUIDE.md
└— EC2_SETUP_GUIDE.md
```

4. PDF Upload & Processing Pipeline

When a user uploads a PDF, the backend executes a strict 7-step pipeline before the document becomes searchable.

Processing steps

1. **File validation** — must be `.pdf` extension, ≤ 50 MB, non-empty filename
2. **Filename sanitisation** — `werkzeug.secure_filename()` strips path traversal characters (`../` , `/etc/passwd` , etc.)
3. **Text extraction** — PyPDF2 reads every page; rejects blank / corrupted PDFs
4. **Medical document validation** — heuristic scorer (see Section 5) rejects non-medical content; file is deleted from disk if rejected
5. **Chunking** — text is split into 500-character chunks with no overlap
6. **Embedding generation** — SentenceTransformers encodes all chunks into 384-dimensional vectors
7. **FAISS indexing** — vectors are added to the conversation's `IndexFlatL2` ; a global chunk-to-document mapping is updated

Upload endpoint — request & response

```
POST /upload
Content-Type: multipart/form-data
```

```
Fields:
  file          (File)      - the PDF
  conversationId (string) - UUID identifying the session
```

```
Response 200:
{
  "message": "File uploaded and indexed successfully",
  "chunks": 42,
  "source": "my_report.pdf",
  "docId": "3f8a1c9e-...",
  "status": "success"
}
```

```
Response 400 (non-medical):
```

```
{  
  "error": "This does not appear to be a medical or health-related document. ..."  
}
```

Response 400 (bad file):

```
{ "error": "Only PDF files are supported" }  
{ "error": "No text could be extracted from PDF" }  
{ "error": "Invalid filename" }
```

Front-end upload flow

- Drag-and-drop or file-picker via `UploadZone` component
- Client-side validation: PDF MIME type check, 50 MB limit
- Upload progress bar driven by `onUploadProgress` axios callback
- Supports multiple documents per conversation (each gets its own UUID)
- After the first upload, UI switches to a compact "Add Another Document" button
- An `AbortController` is wired up so uploads can be cancelled mid-flight

5. Medical Document Validation Layer

NEW FEATURE

`backend/medical_doc_validator.py` — Implemented as a standalone class with zero dependencies beyond the Python standard library. It runs synchronously in the upload pipeline with no LLM calls, no network requests, and no disk I/O.

Design philosophy

A multi-signal heuristic scoring approach is used. Each signal fires independently and contributes a fixed weight to the total score. A document must score ≥ 4 to be accepted. This prevents gaming (a single medical word is insufficient) while still accepting diverse real-world document formats.

Scoring signals

#	Signal	Trigger condition	Weight
1	Lab values with units	Regex matches numbers followed by medical units (mg/dL, mmol/L, IU/L, etc.)	+3 (≥2 hits) / +1 (1 hit)
2	Reference / normal ranges	Patterns like <i>normal: 4.0–10.0, ref range 70–99</i>	+3
3	Patient / sample identifier	Patient ID, MRN, Date of Birth, Accession No, Collection Date, etc.	+3
4	Report header keyword	Lab Report, Discharge Summary, Microbiome Report, IOM Report, etc.	+4
5	Microbiome / gut-health terms	Firmicutes, Lactobacillus, dysbiosis, SCFA, IBS score, GutHeal, etc.	+4 (≥2 hits) / +2 (1 hit)
6	Medical test abbreviations	CBC, HbA1c, TSH, WBC, HDL, LDL, eGFR, CRP, INR, etc.	+2 (≥2 hits) / +1 (1 hit)
7	Medical keyword density	160+ keywords: diagnosis, blood, glucose, cholesterol, hospital, etc.	+1 each, capped at +6
8	Non-medical domain patterns	Invoice numbers, GST codes, legal clauses (<i>whereas, the parties</i>), source code (<code>def func()</code>)	-3 each

Documents accepted (examples)

- IOM SEnS / IBS microbiome kit reports
- Blood panel reports (CBC, CMP, HbA1c, lipid profile)
- Lab test results with reference ranges
- Discharge summaries and clinical notes
- Prescriptions with medication dosages
- Radiology reports
- Health certificates and medical assessments

- Medical research papers

Documents rejected (examples)

- Invoices and purchase orders
- Legal contracts and NDAs
- Source code files exported to PDF
- Business presentations (low medical density)
- CVs and resumes

Analysis window

Only the first **8,000 characters** are analysed — enough to cover the cover page and first section of any real-world document. This keeps validation under 1 ms regardless of PDF length.

NO LLM COST

The validator runs entirely with Python regex. Zero API calls are made for rejected documents. Rejected PDFs are also deleted from disk immediately to avoid orphaned files.

6. RAG Chat Engine

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique where, before calling the LLM, the system first *retrieves* relevant passages from a knowledge base (here, the user's uploaded PDFs) and injects them as context into the LLM prompt. This grounds the answer in real document content and prevents hallucination.

FAISS vector search

- Index type: `faiss.IndexFlatL2` — exact L2 (Euclidean) distance search, no approximation
- Embedding model: `all-MiniLM-L6-v2` — 384 dimensions, excellent semantic understanding
- Top-k: `k = min(5, total_chunks)` — retrieves the 5 most relevant chunks
- Each chunk is 500 characters of raw extracted text
- Source file is tracked for each chunk, enabling citations in the LLM prompt

Context assembly

Retrieved chunks are assembled into a structured context block:

```
[From: blood_report.pdf]
Patient: John Doe  DOB: 12/03/1980
Haemoglobin: 11.2 g/dL  (Reference: 13.0-17.0)
...

---

[From: gut_report.pdf]
IBS Score: 72/100  Firmicutes:Bacteroidetes ratio: 1.8
...
```


LLM prompt structure

The context + question are combined into a structured prompt sent to Bedrock. The prompt instructs the model to:

1. Answer only from the provided context
2. Cite which document a finding comes from
3. Say "I don't have that information in your documents" when the answer isn't in the context
4. Maintain conversational continuity using the last 10 messages of `chatHistory`

Chat endpoint — request & response

```
POST /chat
Content-Type: application/json
```

```
{
  "message":      "What is my IBS score and what does it mean?",
  "conversationId": "uuid-...",
  "chatHistory": [
    { "role": "user",      "content": "... " },
    { "role": "assistant", "content": "... " }
  ]
}
```

```
Response 200:
{
  "response":      "Your IBS score is 72/100, which indicates ...",
  "sources":       ["gut_report.pdf"],
  "chunks_used":   5,
  "documents_used": 1,
  "status":        "success"
}
```

CHAT HISTORY FIX

Prior to the audit, the frontend was sending `files` as the key name instead of `chatHistory` . This meant follow-up questions and conversation context had never worked. Fixed in `App.jsx` .

7. Inference Engine & Query Routing

Before any vector search, every chat message is classified by `InferenceEngine.classify()` (in `llm_config.py`). This routing step prevents wasteful FAISS searches for questions that have nothing to do with the uploaded documents.

Query strategies

Strategy	Triggers on	Action
<code>greeting</code>	"Hi", "Hello", "How are you?"	LLM only — no FAISS search
<code>out_of_domain</code>	Non-health topics (politics, sports, coding, etc.)	LLM-only polite redirect — no FAISS search
<code>general_medical</code> (no doc)	General health question, no document uploaded	LLM answers from training data — no FAISS search
<code>document_query</code>	Question clearly about the uploaded report	FAISS search → context → LLM
<code>hybrid</code>	Combines personal report data + general medical knowledge	FAISS search → context → LLM
<code>follow_up</code>	Short pronoun-based reply to prior message ("What does that mean?")	FAISS search with prior context → LLM

The engine also uses `chat_history` to disambiguate follow-up questions — if the previous exchange was about a specific test result, a follow-up like "Is that normal?" is correctly treated as a document query.

8. Security Guard

All user messages pass through `SecurityGuard.check()` before any processing. It detects prompt injection, jailbreak attempts, and harmful content.

- Runs before FAISS search and before any LLM call
- Returns `{ "safe": false }` for flagged input — request is answered with a soft refusal
- Returns a sanitised version of the message when input is clean
- Flagged inputs are logged server-side with the conversation ID for audit
- The soft refusal response is indistinguishable from a normal response to the user (no error code), preventing information leakage about the filter

9. Multi-Document Management

A single conversation can have multiple PDFs. Users can toggle which documents are active for any given question, and remove documents they no longer need.

How selection works

1. Each document has a boolean `selected` flag (default `true`)
2. When the user toggles a document, `POST /manage-documents` updates the flag
3. The backend immediately rebuilds `combined_index` using only the selected documents
4. All subsequent queries search only the active selection

Combined FAISS index rebuild

```
def rebuild_combined_index(conv_data):  
    # Iterates selected documents in stable order (sorted by doc_id)  
    # Encodes all their chunks together  
    # Creates a single IndexFlatL2 with a global chunk-(doc_id, local_idx) mapping  
    # This mapping is used to attribute each retrieved chunk back to its source file
```

Document Manager UI

- Compact collapsed state by default — shows count badge and number of active docs
- Expands on click to show full list with checkboxes, filename, chunk count, file size
- Checkbox toggle triggers optimistic UI update — immediately reflects the change, reverts on API failure
- Remove button asks for confirmation, then removes the document and rebuilds the index
- Error states auto-clear after 4 seconds

Document list endpoint

```
POST /documents
{ "conversationId": "uuid-..." }

Response:
{
  "documents": [
    { "id": "doc-uuid", "filename": "report.pdf", "size": 18432,
      "selected": true, "chunks": 37 }
  ]
}
```

10. Conversation Isolation

Every browser tab generates a unique `conversationId` (UUID v4) on page load. All backend operations are keyed by this ID.

- User A's FAISS index, document chunks, and chat history are in a separate dict key from User B's
- Conversations cannot read, write, or interfere with each other
- The isolation can be verified at runtime via `POST /debug/isolation-check`
- Conversations persist until the Gunicorn worker is restarted (e.g., after 500 requests via `max_requests`) — there is intentionally no cross-session persistence

MEMORY GROWTH NOTE

`conversation_indices` has no TTL eviction policy. Each conversation occupies RAM proportional to the total size of its document chunks. On a t3.large (8 GB) with the `MemoryMax=4G` systemd ceiling, ~200–300 simultaneous live conversations is a safe upper bound before the worker is automatically killed and restarted by systemd. The `max_requests=500` Gunicorn setting provides a periodic memory-flush.

11. Frontend Application Features

Conversation sidebar

- Unlimited conversations, each with its own title, message history, and file list
- Title auto-set from the first 50 characters of the first user message
- Conversations persisted to `localStorage` — survive page refresh
- Delete conversation button removes from state and `localStorage`
- Sidebar collapses automatically on mobile (≤ 768 px viewport)

Chat interface

- Markdown rendering — assistant responses support bold, italic, lists, code blocks via `react-markdown`
- Auto-scroll to latest message
- Send with Enter, newline with Shift+Enter
- Send button disabled while loading or uploading
- System messages (upload success/failure) displayed inline in the chat timeline
- Chat history (last 10 messages) sent with every request for conversational continuity

Upload zone

- Full drag-and-drop support (DragEvent API)
- Real-time upload progress bar (0–100%)
- Client-side validation before upload: MIME type + 50 MB size check
- After upload: compact "Add Another Document" button replaces the large upload zone

Offline detection

- Listens to `window.addEventListener('online','offline')`
- Red banner appears at top when offline
- Send and upload buttons disabled when offline
- API client also checks `navigator.onLine` before every request

Error handling

- Global error banner with dismiss button for top-level errors
- React `ErrorBoundary` catches render errors without a white-screen crash
- All API calls have automatic retry with exponential backoff (configurable per endpoint)
- Request deduplication — identical concurrent requests share a single in-flight promise

12. API Client & Request Layer

`frontend/src/services/apiClient.js` is the production API client used by all components. It wraps axios with several enterprise-grade features.

Features

Feature	Detail
Retry with exponential backoff	Up to 3 retries (configurable), delay doubles: 1 s → 2 s → 4 s
Request deduplication	Identical concurrent POSTs share one promise; second caller gets same result
Response caching	Optional per-request cache with TTL — disabled for chat/upload by default
Offline guard	Immediately throws "No internet connection" when offline
Input sanitisation	Strips <code><</code> <code>></code> characters from string fields before transmission
FormData upload	Separate <code>postFormData()</code> method with progress callback
Pending count	<code>getPendingCount()</code> exposed to Header component for UI feedback

Routing in development vs. production

Environment	VITE_API_URL	How it works
Local dev (<code>npm run dev</code>)	<code>http://localhost:8000</code>	Direct fetch to Flask dev server; Vite proxy not needed but available
Production build	<code>/api</code>	Relative URL — all calls become <code>/api/chat</code> , <code>/api/upload</code> etc. Nginx rewrites prefix and proxies to Gunicorn on port 8000

13. REST API Endpoints

Method	Path	Description
POST	/upload	Upload and index a PDF
POST	/chat	Send a message, get RAG response
POST	/documents	List documents for a conversation
POST	/manage-documents	Toggle document selection (active/inactive)
POST	/remove-document	Permanently remove a document
POST	/debug/isolation-check	Inspect session state for a conversation ID
OPTIONS	/chat	CORS preflight handler

POST /manage-documents

Request: { "conversationId": "uuid", "docId": "uuid", "selected": true | false }
Response: { "message": "Document selection updated", "docId": "...", "selected": true, "selectedCount": 2 }

POST /remove-document

Request: { "conversationId": "uuid", "docId": "uuid" }
Response: { "message": "Document removed successfully", "docId": "...", "remainingDocuments": 1 }

POST /debug/isolation-check

Request: { "conversationId": "uuid" }
Response: {
 "conversation_id": "uuid",

```
"exists": true,  
"documents_count": 2,  
"selected_documents_count": 1,  
"mapping_status": "74 chunk mappings",  
"total_conversations": 5,  
"documents": [ { "doc_id": "3f8a1c9e...", "filename": "report.pdf",  
                  "chunks": 37, "selected": true } ]  
}
```

SECURITY NOTE

The [/debug/isolation-check](#) endpoint should be removed or gated behind auth before making the deployment publicly accessible to untrusted users. It exposes internal session metadata.

14. Local Development Setup

Prerequisites

- Python 3.10+
- Node.js 18+ and npm
- AWS credentials with Bedrock `InvokeModel` permission

Backend

```
cd virviai-iom-bioworks-gutbotprod/backend

# Create virtual environment
python -m venv venv

# Activate
# Windows: venv\Scripts\activate
# macOS/Linux: source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure credentials
cp .env.example .env
# Edit .env: fill in AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION

# Start the Flask server on port 8000
python app.py
```

Frontend

```
cd frontend
```

```
npm install
npm run dev
# App available at http://localhost:5173
```

VITE PROXY

`vite.config.js` proxies all `/api` requests to `http://localhost:8000` during development, so you can use `VITE_API_URL=/api` in both dev and prod without changing any code.

15. EC2 Production Deployment

Recommended instance sizes

Instance	vCPU	RAM	Users	MemoryMax setting
t3.medium	2	4 GB	~30	2G
t3.large	2	8 GB	~60	4G
t3.xlarge	4	16 GB	~100	8G

Directory structure on EC2

```
/home/ubuntu/gutbot/      ← backend repo (virviai-iom-bioworks-gutbotprod)
├─ backend/
├─ ec2-files/
├─ frontend/              ← frontend repo (uploaded separately)
├   └─ dist/              ← built static files (npm run build)
└─ EC2_SETUP_GUIDE.md
```

One-shot deployment (after EC2 is configured)

```
# 1. Edit ec2-files/nginx-gutbot – set your EC2 IP:
#   server_name 54.123.45.67;

# 2. Edit backend/.env – fill in real AWS credentials

# 3. Run the deploy script:
cd /home/ubuntu/gutbot
```

```
chmod +x ec2-files/deploy.sh  
./ec2-files/deploy.sh
```

The deploy script performs all 9 steps automatically:

1. Pre-flight checks (credentials, IP set, venv exists)
2. Create `/var/log/gutbot/` log directory
3. Install and enable systemd service
4. Install Nginx main config (backs up original)
5. Install Nginx site config, enable site, remove default
6. Test Nginx config syntax (`nginx -t`)
7. Set `.env` file permissions to 600
8. Start Gunicorn service and reload Nginx
9. Live health check — curl to verify backend is responding

SSH safety under load

The systemd service is configured to protect SSH access even under full load:

- `CPUQuota=75%` — GutBot cannot consume more than 75% CPU; SSH always has headroom
- `MemoryMax=4G` (adjust per instance) — hard RAM ceiling
- `MemorySwapMax=0` — disables swap; process is killed rather than degrading indefinitely
- `OOMScoreAdjust=300` — GutBot is killed by the OOM killer before `sshd`
- `Restart=on-failure` — auto-restarts within 5 seconds after a crash
- `WantedBy=multi-user.target` — auto-starts after every reboot

Nginx rate limiting

Zone	Rate	Burst	Applies to
api_limit	10 req/s per IP	20	/chat, /documents, etc.
upload_limit	2 req/s per IP	5	/upload
conn_limit	10 connections per IP	—	All API endpoints

HTTPS (optional but recommended)

```
# After pointing a domain to your EC2 IP:
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d gutbot.yourdomain.com
# Certbot auto-generates and inserts the HTTPS config block
# Auto-renewal: sudo systemctl enable certbot.timer
```

16. Configuration Reference

backend/.env — all fields

Variable	Default	Description
LLM_BACKEND	bedrock	bedrock or llama
AWS_REGION	ap-south-1	AWS region for Bedrock
AWS_ACCESS_KEY_ID	(required)	IAM user key
AWS_SECRET_ACCESS_KEY	(required)	IAM user secret
BEDROCK_MODEL_ID	mistral.mistral-7b-instruct-v0:2	LLM model to use
BEDROCK_MAX_TOKENS	512	Max response tokens
BEDROCK_TEMPERATURE	0.2	Lower = more factual
EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6-v2	HuggingFace embedding model
FLASK_ENV	development	Set to production on EC2
EC2_DOMAIN	(empty)	EC2 IP or domain for CORS whitelist
REQUEST_TIMEOUT	30	Bedrock request timeout (seconds)

frontend/.env.production (built-in)

VITE_API_URL=/api

All production API calls are relative (`/api/chat` , `/api/upload`). Nginx strips the `/api` prefix and forwards to Gunicorn on port 8000.

ec2-files/gunicorn.conf.py — key settings

Setting	Value	Why
<code>workers</code>	<code>1</code>	Session isolation requirement
<code>worker_class</code>	<code>gevent</code>	Non-blocking async concurrency
<code>worker_connections</code>	<code>100</code>	100 simultaneous greenlets
<code>timeout</code>	<code>120</code>	Allow time for large PDF embedding
<code>max_requests</code>	<code>500</code>	Periodic worker restart to prevent RAM leaks
<code>max_requests_jitter</code>	<code>50</code>	Staggers restarts to avoid traffic spike
<code>preload_app</code>	<code>True</code>	Loads embedding model once at startup
<code>bind</code>	<code>127.0.0.1:8000</code>	Localhost only — never exposed to internet

Required AWS IAM permissions

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["bedrock:InvokeModel"],
    "Resource": "arn:aws:bedrock:ap-south-1::foundation-model/*"
  }]
}
```

17. Bug Fixes Implemented

The following bugs were identified and fixed during the production audit:

Bug 1 — Chat history never sent to backend CRITICAL

File: `frontend/src/App.jsx`

Problem: The `/chat` payload used the key `files` instead of `chatHistory`. The backend reads `data.get("chatHistory", [])`. This meant every message was treated as a standalone question — follow-up questions and conversation context never worked at all.

```
// BEFORE (broken):
await apiClient.post("/chat", {
  message: message.trim(),
  conversationId: currentConversationId,
  files: currentConversation?.files || [], // ← wrong key
});

// AFTER (fixed):
const chatHistory = (currentConversation?.messages || [])
  .filter((m) => m.role === "user" || m.role === "assistant")
  .slice(-10)
  .map((m) => ({ role: m.role, content: m.content }));

await apiClient.post("/chat", {
  message: message.trim(),
  conversationId: currentConversationId,
  chatHistory, // ← correct key, correct structure
});
```

Bug 2 — Wrong port throughout frontend HIGH

Files: `.env`, `.env.example`, `vite.config.js`, `services/apiClient.js`, `utils/api.js`

Problem: All fallback URLs used port `5000`. The Flask backend runs on port `8000`. Any time `VITE_API_URL` was unset, all API calls silently went to the wrong port.

```
// BEFORE (broken in 5 files):  
"http://localhost:5000"  
  
// AFTER (fixed):  
"http://localhost:8000"
```

Bug 3 — Production API 404 for all requests HIGH

File: `ec2-files/nginx-gutbot`

Problem: The production frontend sets `VITE_API_URL=/api`, so all calls become `/api/chat`, `/api/upload`, etc. The original Nginx config only matched `/chat` and `/upload` — every production request returned 404 or fell through to the React SPA.

```
# BEFORE (broken):  
location /upload { proxy_pass http://gutbot_backend; }  
location /chat { proxy_pass http://gutbot_backend; }  
  
# AFTER (handles both /upload and /api/upload):  
location ~ ^(/api)?/upload$ {  
    rewrite ^/api(/.*) $1 break;  
    proxy_pass http://gutbot_backend;  
}  
location ~ ^(/api)?/(chat|documents|manage-documents|remove-document|debug) {  
    rewrite ^/api(/.*) $1 break;  
    proxy_pass http://gutbot_backend;  
}
```

Bug 4 — DocumentManager error retry wiped all app state MEDIUM

File: `frontend/src/components/DocumentManager.jsx`

Problem: When fetching the document list failed, the retry button called `window.location.reload()` — a full page reload that wiped all conversations, messages, and files from React state and localStorage was never committed back.

```
// BEFORE (broken):  
const handleRetryFetch = () => {  
    setError(null);  
    window.location.reload(); // ← nuked everything  
};  
  
// AFTER (just refetches documents):  
const fetchDocumentsRef = useRef(null);
```

```
fetchDocumentsRef.current = fetchDocuments;

const handleRetryFetch = useCallback(() => {
  setError(null);
  fetchDocumentsRef.current?.(); // ← only refetches document list
}, []);
```

Bug 5 — start.sh pointed to deleted file and wrong port MEDIUM

File: `backend/start.sh`

Problem: The launch script called `gunicorn app_enhanced:app --port 5000 . app_enhanced.py` had been deleted; port 5000 was wrong. The backend could not be started via this script.

```
# BEFORE (broken):
gunicorn app_enhanced:app --bind 0.0.0.0:5000

# AFTER (fixed):
gunicorn app:app --bind 127.0.0.1:8000 \
  --worker-class gevent --workers 1 \
  -c gunicorn.conf.py
```

Bug 6 — Duplicate null check in DocumentManager LOW

File: `frontend/src/components/DocumentManager.jsx`

Problem: The early-return `if (documents.length === 0 && !isLoading) return null;` appeared twice — once at line 143 and again at line 160. The second check was dead code after JSX refactoring.

18. Security Hardening

Fix 1 — Path traversal vulnerability SECURITY

File: `backend/app.py` — `/upload`

Problem: `file.filename` was used directly in `os.path.join(UPLOAD_FOLDER, file.filename)`. An attacker could upload a file named `../../etc/passwd` or `../app.py` to overwrite arbitrary files on the server.

```
# BEFORE (vulnerable):
filepath = os.path.join(UPLOAD_FOLDER, file.filename)

# AFTER (safe):
from werkzeug.utils import secure_filename
safe_name = secure_filename(file.filename)
if not safe_name:
    return jsonify({"error": "Invalid filename"}), 400
filepath = os.path.join(UPLOAD_FOLDER, safe_name)
```

Fix 2 — CORS production whitelist silently overridden SECURITY

File: `backend/app.py` — `after_request` hook

Problem: A global `@app.after_request` hook unconditionally added `Access-Control-Allow-Origin: *` to every response, overriding the carefully built production whitelist in `flask-cors`. Any origin could read the API in production.

```
# BEFORE (overrides CORS in production):
@app.after_request
def after_request(response):
    response.headers["Access-Control-Allow-Origin"] = "*"    # ← always
    ...

# AFTER (only in development):
@app.after_request
def after_request(response):
    if os.getenv("FLASK_ENV") != "production":
        response.headers["Access-Control-Allow-Origin"] = "*"
```

```
# Production: flask-cors already set the correct whitelist header
...
```

Fix 3 — Nginx Security Headers SECURITY

The Nginx site config adds the following headers to all responses:

- `X-Content-Type-Options: nosniff` — prevents MIME-type sniffing
- `X-Frame-Options: DENY` — prevents clickjacking iframes
- `X-XSS-Protection: 1; mode=block` — legacy XSS filter in older browsers
- `Referrer-Policy: no-referrer` — prevents leaking URL in Referer header

Fix 4 — File permissions on `.env` SECURITY

The deploy script sets `chmod 600 backend/.env` — readable only by the owning user (`ubuntu`), not by other system users or the web server process.

Fix 5 — Gunicorn not exposed to internet

Gunicorn binds to `127.0.0.1:8000` (loopback only). The EC2 Security Group does not open port 8000 to the public. Only Nginx (on port 80/443) is internet-facing, and it rate-limits all requests before forwarding.

Fix 6 — Frontend input sanitisation

`apiClient.js` strips `<` and `>` from all string fields before transmission to prevent reflected XSS via API responses rendered as HTML. The `SecurityGuard` in the backend provides a second layer of prompt-injection filtering.

19. Production Readiness Assessment

This section is the result of a full end-to-end audit performed on the codebase.

Summary verdict

READY FOR PRODUCTION

All critical bugs and security issues have been resolved. The deployment infrastructure (Nginx, Gunicorn, systemd) is production-grade. Two manual steps remain before running `deploy.sh` : set the EC2 IP in `nginx-gutbot` and fill in real AWS credentials in `backend/.env` .

What is ready

Area	Status
Backend Flask/FAISS/Bedrock core	✔ Correct and working
Medical document validation	✔ Implemented and tested
Path traversal vulnerability	✔ Fixed — secure_filename enforced
CORS production override	✔ Fixed — whitelist no longer bypassed
Chat history / conversational context	✔ Fixed — correct key, correct structure
Frontend port mismatches	✔ Fixed in all 5 files
Production Nginx routing (/api/* prefix)	✔ Fixed — regex location blocks with rewrite
EC2 systemd service (auto-start, SSH-safe)	✔ Production-grade config included
Nginx rate limiting, gzip, caching	✔ Configured
start.sh launch script	✔ Fixed — correct file, correct port
DocumentManager retry crash	✔ Fixed — no full-page reload
AWS credentials scrubbed	✔ Only placeholders in repo
Junk and test files removed	✔ Cleaned
Deployment documentation	✔ README, ARCHITECTURE, RUN_GUIDE, EC2_SETUP_GUIDE

Two steps before deploy.sh

1. **Set EC2 IP in `ec2-files/nginx-gutbot` :**
server_name YOUR_EC2_IP_OR_DOMAIN; → server_name 54.x.x.x;
2. **Fill credentials in `backend/.env` :**
Replace YOUR_AWS_ACCESS_KEY_ID and YOUR_AWS_SECRET_ACCESS_KEY with real values.

After that: `./ec2-files/deploy.sh` handles everything automatically.

20. Known Limitations & Future Work

Limitations in current version

1. In-memory session store (no TTL)

`conversation_indices` grows unbounded. Each active conversation holds all chunk embeddings in RAM. On a long-running instance, this will accumulate over time. The `max_requests=500` Gunicorn setting provides a periodic flush (worker restart), but idle conversations are not evicted between flushes.

Mitigation already in place: `max_requests=500` , `MemoryMax` systemd ceiling

Future fix: Add a TTL using `threading.Timer` or switch to Redis for session storage

2. PDF text extraction quality

PyPDF2 extracts raw text. Scanned PDFs (image-only), PDFs with complex column layouts, or PDFs with non-standard encoding may produce garbled or empty text.

Future fix: Add OCR fallback using `pytesseract` or AWS Textract for image PDFs

3. No user authentication

There is no login, no access control, and no persistent user accounts. `conversationId` is generated client-side — anyone who knows a UUID can query that session's documents.

Future fix: Add JWT authentication or AWS Cognito

4. Chunk overlap

Text is split into 500-character non-overlapping chunks. A sentence that straddles a chunk boundary may be split in half, reducing retrieval accuracy for those passages.

Future fix: Use LangChain's `RecursiveCharacterTextSplitter` with 50–100 character overlap

5. Debug endpoint is public

`POST /debug/isolation-check` exposes internal session metadata. Acceptable for internal testing, but should be removed or gated behind auth before a public launch.

6. Single region AWS Bedrock

The app is locked to `ap-south-1` by default. If Bedrock has an outage in that region, the entire chatbot goes down.

Future fix: Implement a fallback region in `llm_config.py`

Possible enhancements

- Streaming responses — Bedrock supports streaming; implementing it would show tokens as they arrive
- Multi-modal support — accept DOCX, images of reports via AWS Textract
- Source highlighting — highlight the exact sentence in the PDF that was retrieved
- Session persistence — save conversation history and documents to DynamoDB or S3
- Feedback loop — thumbs up/down on responses to fine-tune the retrieval strategy
- Admin dashboard — view active sessions, document count, request volume